

# File Organization & Storage

---

## 1. The Physical Reality: Disks and Blocks

---

- Disk storage is organized into **blocks** (pages) — the fundamental unit of transfer between disk and main memory. You can't read/write less than one block at a time, even if you only need a few bytes from it.
  - **Buffering** — the DBMS keeps blocks in in-memory **buffers** while working with them, to avoid repeated disk I/O.
  - **Double buffering** — while one buffer's contents are being processed (or transferred out), the *next* block is already being read into a second buffer concurrently — overlapping computation/output with the next I/O, rather than doing them strictly sequentially. This directly reduces effective access time for continuous, sequential reads (e.g., reading an entire sorted file in order).
- 

## 2. Records and Blocking

---

- **Fixed-length records** — every record is the same size. ✓ Easy to compute any record's exact position. ✗ Can waste space if used to simulate optional/repeating fields (must reserve space for the maximum possible case).
- **Variable-length records** — arise from variable-length fields, repeating fields, optional fields, or a **mixed file** (multiple record types clustered together). Requires extra structure to parse (separator characters, length-prefixed fields, field-name/value pairs, or type codes).

### Blocking factor

$$\text{bfr} = \lfloor B / R \rfloor \quad (B = \text{block size}, R = \text{fixed record size})$$

The number of records that fit in one block;  $B - (\text{bfr} \times R)$  bytes are wasted per block (unless spanning is used).

### Spanned vs. unspanned records

- **Unspanned** — a record must fit entirely within one block; simpler, but wastes the leftover space when  $B$  isn't a perfect multiple of  $R$ .
- **Spanned** — a record is allowed to continue across a block boundary (via a pointer at the end of the first block to the continuation). **Mandatory** when a single record is larger than a block; **advantageous** for variable-length records generally, since it can use up leftover space rather than wasting it.

## Number of blocks needed for a file of $r$ records:

$$b = \lceil r / bfr \rceil$$

## File headers

A metadata structure (per file) storing the disk addresses of the file's blocks and the record format description — this is what lets the system correctly parse records once their block is loaded into a buffer.

## 3. File Organization vs. Access Method

- **File organization** — the *physical layout* of records/blocks on disk (how they're placed and linked).
- **Access method** — the *set of operations* (Find, Insert, Scan, ...) available on top of a given file organization.

**The central goal of file organization:** minimize the number of block transfers needed to locate a desired record — this single goal is the foundation for why indexing and hashing exist at all (developed further in the Indexing topic).

## 4. Unordered Files (Heap Files)

Records stored in **insertion order** — the simplest organization.

| Operation               | Performance                                                                                                                                        |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Insert</b>           | Very fast — append to the last block (its address is tracked in the file header)                                                                   |
| <b>Search</b>           | Very slow — <b>linear search</b> ; average $b/2$ block accesses, worst case $b$                                                                    |
| <b>Delete</b>           | Find record, remove (physically or via a logical <b>deletion marker</b> ), rewrite block; requires periodic reorganization to reclaim wasted space |
| <b>Sorted retrieval</b> | Requires creating an entirely new sorted copy — expensive                                                                                          |

**Direct/relative access:** for fixed-length, unspanned records, the  $i$ -th record can be accessed directly by position — but this only helps if you already know the position; it does **not** help search by field *value* (the file is organized by insertion order, not by content — like a warehouse packing list organized by box number rather than by contents, so finding "the box with a red toaster" still requires opening every box).

## 5. Ordered (Sorted) Files

Records physically arranged on disk by an **ordering field** (an **ordering key**, if that field is also unique).

| Operation                              | Performance                                                                              | Why                                                              |
|----------------------------------------|------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Search on ordering key</b>          | $\log_2(b)$ block accesses — <b>binary search</b>                                        | Records are already in sorted physical order                     |
| <b>Range queries on ordering field</b> | Very efficient                                                                           | Matching records are physically contiguous                       |
| <b>Sequential/next-record access</b>   | Very fast                                                                                | Usually already in the same block                                |
| <b>Insertion</b>                       | <b>Expensive</b> — must find the correct position and shift ~half the records on average | Maintaining physical order requires physically relocating data   |
| <b>Search on a non-ordering field</b>  | Degrades to slow linear search                                                           | Physical order provides no benefit for fields it isn't sorted on |

**Binary search's  $\log_2(b)$  vs. heap's  $b/2$  :** for a file of, say, 1024 blocks, this is the difference between roughly **10** block accesses and roughly **512** — a massive, practically decisive difference for large files.

### Fixing slow insertion — the Overflow (Transaction) File technique

- New records are appended to a separate, unordered **overflow file** instead of being inserted into their correct position in the **main (master) file** immediately.
-  Makes insertion very fast.
-  Searching becomes more complex — a record not found via binary search in the main file requires an additional linear search of the overflow file.
- Periodically, the overflow file is **sorted and merged** back into the master file (reorganization).

## 6. Hashing Techniques

**Core idea:** use a **hash function** to directly **compute** the disk address of a record from its hash field's value — no search at all, ideally just **one** disk access.

- **Hash field** — the field used in the equality search condition.
- **Hash key** — a hash field that's also guaranteed unique.

### Internal hashing — common hash functions

- **Division-remainder (mod):**  $h(K) = K \bmod M$  (most common;  $M$  is often chosen **prime** to improve distribution).
- **Folding** — split the key into parts, combine them (addition/XOR).
- **Digit extraction** — pick specific digits of the key.
- **String conversion** — convert characters to integers via their codes.

### The Collision Problem

Collisions are **inevitable**: the space of possible hash-field values is far larger than the number of actual table slots.

| Resolution method       | How it works                                                                              | Trade-off                                                      |
|-------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| <b>Open addressing</b>  | Probe subsequent slots sequentially until an empty one is found                           | Deletion is complex; clustering degrades performance over time |
| <b>Chaining</b>         | Each slot points to a linked list of colliding records                                    | Simple, efficient insertion/deletion                           |
| <b>Multiple hashing</b> | Try a second/third hash function on collision, falling back to open addressing eventually | More computation per collision, but spreads collisions better  |

**Load factor** =  $r/M$  (records / table size) — keep it around **70–90%** for good performance (too high → excess collisions; too low → wasted memory).

### External Hashing (for disk files)

- The unit is a **bucket** (one disk block, or a cluster of contiguous blocks) — not a single slot.
- Multiple records can hash to the same bucket without immediate collision, until the bucket **fills up**; then **overflow buckets** chain together off the primary bucket.
- ☒ Fastest possible retrieval by hash-field equality (often just 1 disk access).

- ❌ **Static hashing problem** — a fixed number of buckets  $M$  doesn't adapt as the file grows: too few buckets → long overflow chains (performance collapses); too many → wasted space.
- ❌ Searching by a **non-hash** field degrades to a slow linear scan, same as a heap file.

## Dynamic Hashing — fixing the static-size problem

**Extendible Hashing** — a **directory + buckets** two-level structure:

- **Directory** — an array of  $2^d$  pointers ( $d$  = **global depth**); the first  $d$  bits of a record's hash value index into it to find the target bucket.
- Each **bucket** has its own **local depth**  $d'$  — how many hash bits currently distinguish records belonging to *that specific* bucket.
- **On overflow**: split the bucket, increment its local depth  $d'$ , rehash its records using one more bit.
- **Directory doubling**: if the overflowing bucket's local depth already **equals** the global depth ( $d' = d$ ), the directory itself is too small to give the two new buckets distinct entries — so the **directory doubles** ( $d$  increments), giving the needed extra addressing granularity.
- ✅ No overflow chains, so no performance collapse; **localized reorganization** (a split affects only the bucket involved, never triggers a full-file reorganization); directory overhead is small (fits comfortably in memory).
- ❌ Retrieval typically needs **2** disk accesses (directory + bucket) instead of static hashing's ideal 1 — though the directory is usually cached in RAM, largely absorbing this cost.

**Dynamic Hashing (tree-based)** — a precursor/alternative to extendible hashing, using a **binary tree** instead of a flat directory array: internal nodes route left/right per hash bit, leaf nodes point to buckets. Functionally similar outcome, different directory *organization* (tree vs. flat array).

**Linear Hashing** — grows the file **without any directory at all**:

- Buckets split in a **fixed, predetermined linear order** (0, 1, 2, 3, ...) — **regardless of which bucket actually overflowed**. An overflow anywhere just triggers splitting *the next bucket in line* (tracked by a **split pointer**  $n$ ).
- Uses **two hash functions at once** during a growth phase:  $h_i(K) = K \bmod M$  (old) and  $h_{i+1}(K) = K \bmod 2M$  (new) — a search checks which function applies based on whether the target bucket number is before or after the current split pointer.
- ✅ **No directory at all** — eliminates that space/indirection overhead entirely; splits can be triggered by overall **load factor** rather than reactively per-overflow, allowing tunable performance; can also **shrink** (merge buckets) symmetrically.
- ❌ More conceptually intricate to reason about (which hash function applies depends on the current split-pointer position, not just the key).

**Interview soundbite:** "Extendible hashing pays a small, predictable extra directory-lookup cost in exchange for perfectly localized splits and no overflow chains. Linear hashing avoids the directory's overhead entirely by giving up the *freedom* to split whichever bucket actually overflowed — it always splits the next one in a fixed sequence instead, which sounds odd until you see that the two-hash-function trick guarantees this still converges correctly over time."

## 7. Mixed Files & Alternative Physical Organizations

---

### Mixed files (physical clustering)

Store records of **different, related entity types together**, physically contiguous on disk — e.g., a **Department** record immediately followed by all its related **Student** records.

- Each record carries a **record type field** as its first field, so the DBMS knows how to interpret the rest (via the system catalog).
- **Why:** dramatically reduces disk I/O for a common access pattern like "retrieve a department and all its students" — one contiguous read instead of separate lookups jumping around the disk.
- Common in legacy hierarchical/network DBMSs, and object DBMSs (for clustering related complex objects).

### Column-based storage

Instead of storing all fields of one record together (row-store), store **all values of one column together** (column-store) — a radical alternative layout.

- ☒ Extremely efficient for **analytical** queries that aggregate over a few columns across millions of rows (e.g., "average salary") — only the relevant column's blocks need to be read at all.
- ☒ Very inefficient for **transactional** workloads needing to insert/update a whole row at once (that one logical row is now scattered across many separate column blocks).

This exact row-store vs. column-store trade-off is the foundational distinction between OLTP databases (row-store, e.g., typical MySQL/PostgreSQL usage) and OLAP/data-warehouse systems (column-store, e.g., many analytics engines).

---

## 8. RAID — Parallelizing Disk Access

---

### The problem RAID solves

CPU/RAM performance has improved far faster than disk access time over the decades — disk I/O is a persistent bottleneck, and reliability *decreases* as you add more individual disks (more components = more failure points, since MTBF of an array roughly divides by the number of disks).

## The core technique: Data Striping

Split data into segments and distribute them **across multiple disks**, so multiple disks can be read/written **in parallel**.

- **Bit-level striping** — every byte's individual bits spread across disks; all disks participate in every I/O; best for large sequential transfers.
- **Block-level striping** — whole blocks distributed round-robin across disks; better for parallelizing many small, independent requests.

## The two RAID goals

| Goal        | Technique                                                |
|-------------|----------------------------------------------------------|
| Performance | Striping (parallel I/O)                                  |
| Reliability | Redundancy (mirroring, or parity/error-correcting codes) |

**Reconstructing from parity (the core trick):** if  $A \oplus B \oplus C = \text{Parity}$ , and disk  $C$  fails, then  $C = A \oplus B \oplus \text{Parity}$  — the lost disk's data is recovered purely from the surviving disks and the parity, without ever needing a full duplicate copy.

## RAID Levels

| Level    | Technique                                               | Striping | Redundancy     | Fault tolerance                               | Storage efficiency | Best for                                                    |
|----------|---------------------------------------------------------|----------|----------------|-----------------------------------------------|--------------------|-------------------------------------------------------------|
| <b>0</b> | Striping only                                           | Block    | None           | <b>0 disks</b> — any failure loses everything | 100%               | Non-critical, performance-only data (cache, scratch space)  |
| <b>1</b> | Mirroring                                               | None     | Full duplicate | 1 disk                                        | 50%                | Critical systems (banking, logs, OS drives)                 |
| <b>2</b> | Bit-level + Hamming ECC                                 | Bit      | ECC disks      | 1 disk                                        | Low                | Obsolete/theoretical                                        |
| <b>3</b> | Byte-level striping + dedicated parity disk             | Byte     | 1 parity disk  | 1 disk                                        | $(N-1)/N$          | High-throughput sequential (video/audio editing)            |
| <b>4</b> | Block-level striping + dedicated parity disk            | Block    | 1 parity disk  | 1 disk                                        | $(N-1)/N$          | Rare today — parity disk is a write bottleneck              |
| <b>5</b> | Block-level striping + <b>distributed</b> parity        | Block    | Distributed    | 1 disk                                        | $(N-1)/N$          | <b>Most popular general-purpose RAID</b> — file/web servers |
| <b>6</b> | Block-level striping + <b>double</b> distributed parity | Block    | Double         | <b>2 disks</b>                                | $(N-2)/N$          | Mission-critical, large enterprise storage                  |

**Why RAID 5 beats RAID 4 despite otherwise being similar:** RAID 4's single dedicated parity disk becomes a **write bottleneck** — every single write anywhere in the array must also update that one parity disk. RAID 5 spreads parity blocks across **all** disks, so no single disk is disproportionately

hammered by every write — this single change is what made RAID 5 the practical default over RAID 4.

## Hybrid RAID levels

- **RAID 0+1** — a mirror of two striped sets (fast, and redundant, but a whole striped set must be rebuilt if even one disk in it fails).
- **RAID 1+0 (RAID 10)** — a stripe across multiple mirrored pairs — generally preferred over 0+1, since only the single failed disk's mirror pair needs rebuilding, not an entire striped set.
- **RAID 50 / 60** — multiple RAID 5 (or 6) arrays, themselves striped together — used for very large-scale storage needing both RAID 5/6's balance *and* extra raw throughput.

**Interview soundbite:** "RAID's whole design space is a trade-off triangle between performance, redundancy, and storage efficiency. RAID 0 maximizes performance and efficiency but has zero redundancy. RAID 1 maximizes simplicity and redundancy but halves your usable capacity. RAID 5 is the practical sweet spot — good performance, only one disk's worth of capacity sacrificed to parity, and it tolerates exactly one failure — which is exactly why it's the default choice for most general-purpose server storage."

---

## Interview Questions With Answers

---

### Q1. Why does a heap file offer fast insertion but slow search, while a sorted file offers the opposite trade-off?

**Answer:** A heap file simply appends new records to the end of the last block, requiring no search for a correct position — insertion is nearly free. But because records aren't organized by any searchable criterion, finding a specific record requires scanning through blocks linearly, averaging  $b/2$  block accesses. A sorted file, by contrast, physically arranges records by an ordering key, enabling binary search ( $\log_2(b)$  accesses) for lookups on that key — but maintaining that physical order means inserting a new record requires finding its correct position and shifting roughly half the file's records on average, making insertion expensive.

### Q2. How does the overflow (transaction) file technique mitigate a sorted file's slow insertion, and what does it cost in return?

**Answer:** Instead of inserting a new record directly into its correct sorted position in the main file (which requires shifting existing records), new records are appended to a separate unordered overflow file — a fast, heap-file-style insertion. The cost is that searching becomes two-phase: a binary search of the sorted main file, and if the record isn't found there, an additional linear search of the overflow file — periodically, the overflow file is sorted and merged back into the main file during reorganization to keep this overhead from growing unbounded.

### **Q3. Why is external hashing's basic (static) scheme problematic for a file that grows over time?**

**Answer:** Static hashing fixes the number of buckets  $M$  when the file is created. If the file later grows well beyond what  $M$  buckets can comfortably hold, records increasingly collide into the same buckets, forcing longer and longer overflow chains — search performance degrades toward that of a heap file's linear scan, defeating the entire point of hashing. If  $M$  was instead chosen too large upfront to avoid this, significant space sits wasted for a file that hasn't yet grown into it. Static hashing has no mechanism to adapt  $M$  as the actual data volume changes.

### **Q4. Explain how extendible hashing avoids the overflow-chain problem that static hashing suffers from.**

**Answer:** Instead of a fixed bucket count, extendible hashing uses a directory of  $2^d$  pointers (global depth  $d$ ) that can grow, plus per-bucket local depths tracking how many hash bits currently distinguish that bucket's records. When a bucket overflows, it's split (not chained) — its local depth increases and its records are rehashed using one more bit; if the bucket's local depth was already equal to the global depth, the directory itself doubles to provide enough addressing granularity for the split. Because overflowing buckets are split rather than chained, retrieval never has to walk a growing overflow chain — it stays at essentially a fixed cost (directory lookup, then one bucket read) regardless of how much the file has grown.

### **Q5. Why does linear hashing not need a directory at all, unlike extendible hashing?**

**Answer:** Linear hashing splits buckets in a fixed, predetermined linear order (0, 1, 2, 3, ...) tracked by a single split pointer, regardless of which specific bucket actually triggered an overflow — this predictability means a record's bucket location can always be computed directly from its hash value and the current split pointer position (checking whether to apply the old or new hash function), without needing an auxiliary directory structure to track irregular, per-bucket split history the way extendible hashing's directory does. The trade-off is that linear hashing gives up the flexibility to split precisely the bucket that overflowed — it has to wait for the fixed split order to eventually reach that bucket, relying on the "meanwhile" overflow chains from the mismatch to correctly redistribute once split.

### **Q6. Why is a mixed file (physical clustering of related record types) beneficial for some queries but not a general-purpose default?**

**Answer:** Physically clustering related records together (e.g., a department immediately followed by all its students) means a common access pattern like "retrieve this department and all its students" can be satisfied with one contiguous disk read, instead of separately locating the department record and then jumping around the disk to find each student record individually — a large I/O savings for that specific relationship. It's not a general-purpose default because it optimizes specifically for one access pattern at the potential expense of others — a query that needs, say, "all departments" without their students would now have to skip over interleaved student records it doesn't need, and the

physical clustering assumption breaks down if the actual query workload doesn't consistently favor that one relationship.

### **Q7. Why is column-based storage efficient for analytical queries but poor for transactional workloads?**

**Answer:** Column-store physically groups all values of a single column together on disk. An analytical query aggregating over a few columns across millions of rows (e.g., average salary) only needs to read the blocks for those specific columns, skipping everything else entirely — a huge I/O saving. A transactional workload needing to insert or update one complete row, by contrast, now has that single logical row's data scattered across many separate column-specific locations on disk, requiring many separate writes (one per column touched) instead of one contiguous write — exactly the opposite of what row-store naturally provides for that access pattern.

### **Q8. Why is RAID 5 generally preferred over RAID 4, given they're otherwise very similar (both block-level striping with parity)?**

**Answer:** RAID 4 stores all parity information on one single dedicated disk, meaning every write operation anywhere in the array must also update that one parity disk — making it a write bottleneck that limits overall write throughput regardless of how many data disks are available. RAID 5 distributes parity blocks across all disks in the array instead of dedicating one, so the parity-update load is spread evenly rather than concentrated on a single disk — this removes the bottleneck while providing the exact same fault tolerance (survives one disk failure) and the same storage efficiency, which is why RAID 5 essentially superseded RAID 4 as the practical general-purpose choice.

### **Q9. Scenario: A company is choosing RAID for two different systems — one is a read-heavy web server farm's shared file storage, and the other is a mission-critical financial ledger database that absolutely cannot tolerate data loss even during a second failure while recovering from a first. Recommend a RAID level for each, and justify why the first company's choice wouldn't be adequate for the second.**

**Answer:** For the read-heavy web server file storage, **RAID 5** is a strong fit — it provides good parallel read performance, tolerates one disk failure, and only sacrifices one disk's worth of capacity to parity (efficiency  $(N-1)/N$ ), which is an appropriate balance when the workload is read-heavy and a single-failure tolerance is an acceptable risk level. For the mission-critical financial ledger, RAID 5 would be inadequate specifically because of the failure window during recovery: while RAID 5 rebuilds a failed disk from parity, the array is running in a degraded state, and a **second** disk failure during that rebuild window would cause total data loss — RAID 5 has zero tolerance for two simultaneous/overlapping failures. **RAID 6** (or RAID 10 for even better performance during rebuilds) is the appropriate choice here, since RAID 6's double distributed parity explicitly tolerates two disk failures at once, directly addressing the requirement that a second failure during recovery from the first must not cause data loss — at the cost of slightly more storage overhead  $((N-2)/N)$  and additional parity-computation overhead on writes, both of which are acceptable trade-offs given the stated priority on reliability over raw efficiency.